

Assignment 07

Ganesh Gundekarla

11700551

Code explanation:

Data Loading and exploration of the data :

- a. Here, the importing of the required library like pandas is done in order to import the CSV dataset into a dataframe named df .
- b. `.head()` is used to display the first few rows.
- c. `Df.info()` is used to provide us with some information regarding the data frame which also includes data types and the possible non null counts.
- d. `.counts()` is used to count the possible occurrences of a set of values of the given columns .
- e. Then the null values are being calculated totally by using `'df.isNull().sum()'` , then after this, we then fill the missing values by calculating the `'missing_percentage'` which calculates the mean of each column then multiply it by 100 .

Data Preprocessing:

- a. In the preprocessing step , the missing values are then filled using the `df_filled` in the code .Then , `df_encoded` is formed freshly using the getting the dummy values , which is used for the sole purpose of converting the categorical variables into the indicator var's which is a technique named as `'one-hot -encoding'`.
- b. This would result in some extra categorical columns ,with some values.

Splitting the preprocessed data into the two Training , Testing sets.

- a. Then the process of split the data into features and a target variable. The `train_test_split` which is imported from `'sklearn.preprocessing'` which is solely used to select features .
- b. Then the training sets are scaled.

Model Training and Evaluation:

- a. The `'kNeighborsRegressor'` from the `'sklearn.neighbors'` which is used to create the kNN regressor model with the neighbors value set to 5 .

- b. This model is then fitted with test data using knn_regressor. The fit method is taking the parameters of fx_train, y_train and note that the xtrain data is scaled.
- c. Then predictions are made using the equation

$$Y_{\text{prediction}} = \text{knn_regressor.predict}(x_{\text{test_scaled}})$$
predicts data on the test one which is trained using the trained dataset.
- d. Then , mean_squared_error is being calculated from the sklearn.metrics to perform mse between true and the predicted values.
- e. R-squared val is also being calculated representing variance .

Code for knn : ‘

```
import pandas as pde
df1= pde.read_csv('dataset.csv')
df1.head()
df.info()
df1['State'].value_counts()
df1.isnull().sum()
missing_percentage = df1.isnull().mean() * 100
print(missing_percentage)
df_filled.isnull().sum()
df_encods = pd.get_dummies(df_filled, prefix='Category')

print(df_encods)
y = df_encods['Area Planted (acres)']
X = df_encods.drop('Area Planted (acres)', axis = 1)
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.33,
random_state=42)
from sklearn.preprocessing import StandardScaler

scalero= StandardScaler()
X_train_scaled = scalero.fit_transform(X_train)
X_test_scaled = scalero.transform(X_test)
```

```

from sklearn.neighbors import KNeighborsRegressor
knn_regressor = KNeighborsRegressor(n_neighbors=5)
knn_regressor.fit(X_train_scaled, y_train)
from sklearn.metrics import mean_squared_error
y_prediction = knn_regressor.predict(X_test_scaled)
mse = mean_squared_error(y_test, y_prediction)
print(f'Mean Squared Error: {mse}')
from sklearn.metrics import r2_score
r_squared = r2_score(y_test, y_pred)
print(f'R-squared: {r_squared}')

```

KNN.ipynb ☆

File Edit View Insert Runtime Tools Help All changes saved

+ Code + Text

RAM  Disk  Colab AI ^

```
[2] import pandas as pd
```

```
[3] df = pd.read_csv('dataset.csv')
df.head()
```

	State	Year	Nitrogen (%)	Nitrogen (Pounds/Acre)	Phosphorous (%)	Phosphorous (Pounds/Acre)	Potash (%)	Potash (Pounds/Acre)	Area Planted (acres)	Harvested Area (acres)	Harvested Area (Pounds/Acre)
0	Alabama	1964	99.0	72.0	100.0	62.0	99.0	63.0	NaN	NaN	NaN
1	Alabama	1965	100.0	81.0	100.0	63.0	100.0	66.0	NaN	NaN	NaN
2	Alabama	1966	100.0	83.0	100.0	69.0	100.0	70.0	NaN	NaN	NaN
3	Alabama	1967	100.0	78.0	100.0	71.0	100.0	73.0	NaN	NaN	NaN
4	Alabama	1968	100.0	71.0	99.0	71.0	99.0	73.0	NaN	NaN	NaN

Next steps: [Generate code with df](#) [View recommended plots](#)

```
[4] df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 756 entries, 0 to 755
Data columns (total 11 columns):
#   Column              Non-Null Count  Dtype
---  ---
0   State                756 non-null   object
1   Year                 756 non-null   int64
2   Nitrogen (%)         468 non-null   float64
3   Nitrogen (Pounds/Acre) 468 non-null   float64
```

KNN.ipynb ☆
File Edit View Insert Runtime Tools Help All changes saved

+ Code + Text
Name: State, dtype: int64

[7] df.isnull().sum()

State	0
Year	0
Nitrogen (%)	288
Nitrogen (Pounds/Acre)	288
Phosphorous (%)	288
Phosphorous (Pounds/Acre)	288
Potash (%)	294
Potash (Pounds/Acre)	298
Area Planted (acres)	126
Harvested Area (acres)	126
Lint Yield (Pounds/Harvested Acre)	126

dtype: int64

[8] df.shape

(756, 11)

[9] # Calculate the percentage of missing values for each column
missing_percentage = df.isnull().mean() * 100

Print the percentage of missing values
print(missing_percentage)

State	0.000000
Year	0.000000
Nitrogen (%)	38.095238
Nitrogen (Pounds/Acre)	38.095238
Phosphorous (%)	38.095238
Phosphorous (Pounds/Acre)	38.095238
Potash (%)	38.888889
Potash (Pounds/Acre)	39.417989
Area Planted (acres)	16.666667
Harvested Area (acres)	16.666667
Lint Yield (Pounds/Harvested Acre)	16.666667

dtype: float64

[10] column_means = df.mean()

Fill missing values with the mean of their respective column
df_filled = df.fillna(column_means)

<ipython-input-10-6da0f675c1f9>:1: FutureWarning: The default value of numeric only in DataFrame.mean is deprecated.

```
KNN.ipynb ☆
File Edit View Insert Runtime Tools Help All changes saved
+ Code + Text
[13] df_encoded.head()
```

	Year	Nitrogen (%)	Nitrogen (Pounds/Acre)	Phosphorous (%)	Phosphorous (Pounds/Acre)	Potash (%)	Potash (Pounds/Acre)	Area Planted (acres)	Harvested Area (acres)	Lint Yield (Pounds/Harvested Area)
0	1964	99.0	72.0	100.0	62.0	99.0	63.0	874.279365	778.571429	772.41
1	1965	100.0	81.0	100.0	63.0	100.0	66.0	874.279365	778.571429	772.41
2	1966	100.0	83.0	100.0	69.0	100.0	70.0	874.279365	778.571429	772.41
3	1967	100.0	78.0	100.0	71.0	100.0	73.0	874.279365	778.571429	772.41
4	1968	100.0	71.0	99.0	71.0	99.0	73.0	874.279365	778.571429	772.41

5 rows x 24 columns

```
[14] df_encoded.isnull().sum()
```

Year	0
Nitrogen (%)	0
Nitrogen (Pounds/Acre)	0
Phosphorous (%)	0
Phosphorous (Pounds/Acre)	0
Potash (%)	0
Potash (Pounds/Acre)	0
Area Planted (acres)	0
Harvested Area (acres)	0

```
KNN.ipynb ☆
File Edit View Insert Runtime Tools Help All changes saved
+ Code + Text
[11] df_filled.isnull().sum()
```

State	0
Year	0
Nitrogen (%)	0
Nitrogen (Pounds/Acre)	0
Phosphorous (%)	0
Phosphorous (Pounds/Acre)	0
Potash (%)	0
Potash (Pounds/Acre)	0
Area Planted (acres)	0
Harvested Area (acres)	0
Lint Yield (Pounds/Harvested Area)	0

dtype: int64

```
[12] # Convert categorical variable into dummy/indicator variables
df_encoded = pd.get_dummies(df_filled, prefix='Category')

print(df_encoded)
```

752	2014	89.521368	87.205128	65.450855
753	2015	66.000000	65.000000	46.000000
754	2016	89.521368	87.205128	65.450855
755	2017	68.000000	77.000000	52.000000

	Phosphorous (Pounds/Acre)	Potash (%)	Potash (Pounds/Acre)	\
0	62.000000	99.000000	63.000000	
1	63.000000	100.000000	66.000000	
2	69.000000	100.000000	70.000000	
3	71.000000	100.000000	73.000000	

KNN.ipynb ☆

File Edit View Insert Runtime Tools Help All changes saved

+ Code + Text

✓ RAM Disk Colab AI

Category_Texas
dtype: int640

[15] y = df_encoded['Area Planted (acres)']
X = df_encoded.drop('Area Planted (acres)', axis = 1)

[16] from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.33, random_state=42)

[17] from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

[18] from sklearn.neighbors import KNeighborsRegressor
knn_regressor = KNeighborsRegressor(n_neighbors=5)
knn_regressor.fit(X_train_scaled, y_train)

▼ KNeighborsRegressor
KNeighborsRegressor()

[19] from sklearn.metrics import mean_squared_error
y_pred = knn_regressor.predict(X_test_scaled)

Evaluate the model
mse = mean_squared_error(y_test, y_pred)
print(f'Mean Squared Error: {mse}')

Mean Squared Error: 59825.5759434707

⬆ ⬇ 🔗 💬 ⚙️ 📄 🗑️ ⋮

▶ from sklearn.metrics import r2_score
r_squared = r2_score(y_test, y_pred)
print(f'R-squared: {r_squared}')

R-squared: 0.9598209871417859

[] Start coding or generate with AI.

ScreenShot